

[Home](#)
[Magazine](#)
[Live Events](#)
[News](#)
[Forums](#)
[Store](#)
[Directory](#)
[Editorial](#)
[Advertising](#)
[User/Login](#)
[Contact](#)
[Talk to us](#)

You are here



Showcase



Volume Number: 13 (1997)

Issue Number: 2

Column Tag: Programmer's Challenge

Programmer's Challenge

By Bob Boonstra, Westford, MA

Othello™

This month's Challenge is going to be another round-robin competition for a well-known board game - this time the game of Othello. The classic game of Othello is played on an 8x8 board using discs that are black on one side and white on the other side. The game starts with four discs in the center squares of the board, two black discs on diagonally adjacent squares, and two white discs on the other diagonally adjacent squares. Players alternate placing an additional disc, with black moving first. A move consists of "outflanking" one or more of your opponent's discs. Outflanking means placing a disc so that at least one row of your opponent's discs is bordered by a disc of your color, including the disc just placed on the board. A row is defined as one or more discs of a single color in a continuous straight horizontal, vertical, or diagonal line. When a player moves, the row or rows of outflanked discs are flipped over showing his or her color. If a player cannot outflank a disc, the turn is forfeited and the opponent takes another turn. The game is over when the board is filled or neither player can move. The player with the most discs showing is the winner.

In this Challenge, the game will be generalized to boards larger than 8x8. The prototype for the code you should write is:

```
Boolean /*legalMove*/ Othello (
    long boardSize, /* number of rows/columns in the game board */
    long oppRow,    /* row where opponent last moved, 0 .. boardSize-1 */
    long oppCol,    /* column where opponent last moved, 0 .. boardSize-1 */
    long *moveRow,  /* return your move - row, 0 .. boardSize-1 */
    long *moveCol,  /* return your move - column, 0 .. boardSize-1 */
    void *privStorage, /* preallocated storage for your use */
    long storageSize, /* size of privStorage in bytes */
    Boolean newGame, /* TRUE if this is your first move in a new game */
    Boolean playBlack /* TRUE if you play first (black pieces) */
);
```

For your first move, Othello will be called with newGame set to TRUE. The size of the board, an even number between 8 and 64, will be provided in boardSize. On your first move you should initialize the board with white tiles at (row,col) = (boardSize/2-1,boardSize/2-1) and (boardSize/2,boardSize/2), and black tiles at (boardSize/2-1,boardSize/2) and (boardSize/2,boardSize/2-1). Rows and columns are numbered from 0 to boardSize-1. If playBlack is TRUE, you are to play the black pieces, and therefore play first. Otherwise, you play the white pieces. Your code and the code of an opponent will alternate play. Your opponent's move will be provided in (oppRow, oppCol), which will be set to (-1,-1) if your opponent is unable to move or if you are moving first. When your code is called you should flip your tiles that were outflanked by your opponent's move, calculate your own move, and store it in (*moveRow, *moveCol). If you are unable to move, store the values (-1,-1). If for any reason you believe your opponent has made an illegal move, Othello should return a value of FALSE, otherwise it should return TRUE.

Your code will be provided with storageSize bytes of preallocated storage (at least 1 MB) pointed to by privStorage. This storage will be pre-initialized to zero before your first move

THIS MONTH'S ISSUE

TODAY'S DEAL

and will persist between moves. You should not allocate any additional dynamic storage beyond that provided by `privStorage`. Small amounts of static storage are permitted.

The Challenge will be scored by a tournament where each entry plays against each other entry twice for each of a number of board sizes, once playing the black pieces and once playing the white pieces. In the event that a large number of entries are received, another fair tournament schedule may be used. The score will be based on the margin of victory (or loss) and the execution time used to compute the moves. For each game, the score will be computed by

```
[(# of player's pieces showing - # of opponent's pieces showing) - (execution time in seconds)/30] / (boardSize*boardSize)
```

The player with the highest score for all games in the tournament will be the winner.

Those of you needing more information about the rules of the game can check out your local toy store, or look at <http://www.daimi.aau.dk/~tusk/pbmserv/othello/othello.rules.html>. Other information can be found at <http://www.armory.com/~iioa/othguide.html>, the International Internet Othello Association page.

Othello is a registered trademark of Tsukuda Original, licensed by Anjar Co., copyright 1973, 1990 by Pressman Toy Corporation.

Three Months Ago Winner

Congratulations to **Thomas Studer** (Syracuse, N.Y.) for submitting the winning entry to the Router Rules Challenge. The problem was to generate a set of (mask, value, allow/deny) triplets that could be used by a router to allow net access to a specified set of subnet addresses. Given a subnet address, the rules would be scanned in sequence by the router, and the first rule to fire would determine whether access was allowed or denied. A rule fires when the subnet address, logically ANDed with the rule mask, is equal to the rule value. Solutions were to minimize a score that was the sum of two quantities, the number of rules generated, and the time taken to generate those rules (in half-seconds).

Of the six solutions I received, only two of them worked correctly for my test cases. The other four either generated rules that produced incorrect results for some input subnet values or had not completed execution after running overnight. Thomas' winning solution assigns each input value to a group determined by the number of bits set in each "chunk" of 1, 2, 4, or 8 bits. The code then uses this mapping to search for "buddy" input values that differ in only one bit position. When such a "buddy" value is found, the values are combined, and a mask is updated to indicate which bits should be masked out in a router rule. The code makes use of three large pre-built `BitGrpMapper` tables that allow Thomas to calculate the number of bits set in each chunk of 2, 4, or 8 bits. For each chunk of size *n*, these tables represent the number of bits set in each input value as a base *n*+1 number, an interesting and compact representation.

The second-place solution by **Alan Hart** used a recursive technique to generate the Karnough map representing the allowed values. While this approach generated more rules (and therefore a poorer score) than the winning solution, it ordered the rules more optimally, in that rules governing a larger number of allowed subnets occurred earlier. This was not a criterion for scoring this Challenge, but it would be an important real-world consideration.

More compact rule sets than those generated by the two correct solutions might be possible. The very long-running solution mentioned above appeared to generate a small number of rules, half as many in some cases, at significant execution time expense.

The table below provides the language, code size, and data size for each of the six entries received. For the two correct entries, it also provides the score (based on rules generated and execution time), the average number of rules scanned before an allowed subnet value is granted access (summed over all test cases), and the total number of rules generated for all test cases.

Name Language Score Avg Rules Rules Gen Code Data

Thomas Studer	C++	3365	1414	3365	3844	12824
Alan Hart	C	4716	1193	4715	1472	148
E. M.	C++ *	6308	6672			
L. N.	C *	4024	220			
K. S.	C *	2472	8			
M. Y.	C++ *	3312	299			

TOP 20 CONTESTANTS

Search 

Community Search: 

MacTech Search: 

Software Updates via MacUpdate

Latest Forum Discussions 

iPHONE APP SHOWCASE

Price Scanner via MacPrices.net

Jobs Board

Here are the Top 20 Contestants for the Programmer's Challenge. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants.

Rank Name Points

1. Munter, Ernst 195
2. Gregg, Xan 114
3. Larsson, Gustav 87
4. Lengyel, Eric 40
5. Lewis, Peter 32
6. Boring, Randy 27
7. Cooper, Greg 27
8. Antoniewicz, Andy 24
9. Beith, Gary 24
10. Kasparian, Raffi 22
11. Cutts, Kevin 21
12. Nicolle, Ludovic 21
13. Picao, Miguel Cruz 21
14. Brown, Jorg 20
15. Gundrum, Eric 20
16. Studer, Thomas 20
17. [Name deleted] 20
18. Karsh, Bill 19
19. Mallett, Jeff 17
20. Nevard, John 17

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to suggest a Challenge that I use. The points you can win are:

1st place 20 points 5th place 2 points

2nd place 10 points finding bug 2 points

3rd place 7 points suggesting Challenge 2 points

4th place 4 points

Here is Thomas' winning solution:

RouterRules.cp

Copyright © 1996 Thomas Studer

```
// Memory requirements
// -----
// About 12K of static tables. A variable amount of heap memory. The program will run // faster and/or yield
// compacter results given more memory (in most cases). The
// maximum amount of dynamic memory is approximately (chunkSize+1)^chunkCount // times the size of
// a pointer + the number of input values times 12 bytes (the size of
// BitGrpEntry). 'chunkSize' is 1, 2, 4 or 8. 'chunkCount' is the width (in bits) of the
// input values / chunkSize, rounded up. 'chunkCount' is between 4 and 32, inclusive.
//
// How it works
// -----
// - The values to be reduced are arranged in memory in a number of linked lists. The
// program then loops through these lists trying to find values (with matching masks)
// that differ in exactly one bit ('buddy' values). The value in a pair of such compatible // values that has
// a '1' in the only differing bit position is thrown away and the mask of // the the remaining value is updated
// by clearing that bit. If, for a given value no
// 'buddy' can be found, it is output and the program continues until all the internal
// lists are empty.
```